

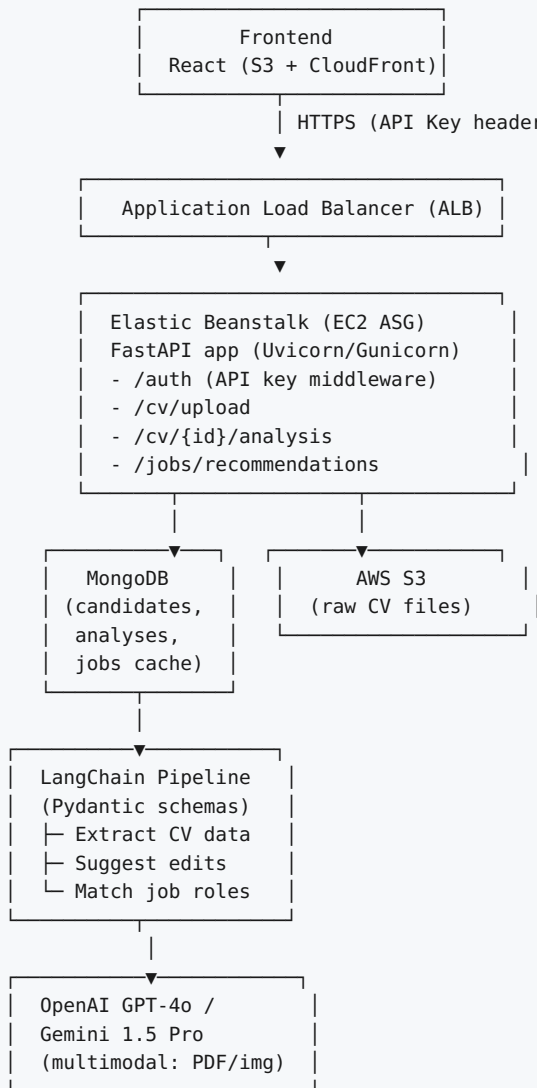
- [1. Project Overview](#)
- [2. Architecture](#)
- [3. Folder Structure](#)
- [4. Data Models \(MongoDB\)](#)
- [5. LLM Output Schemas \(Pydantic — Python’s Zod\)](#)
- [6. LangChain Service \(Multimodal Extraction + Structured Output\)](#)
- [7. API Endpoints](#)
- [8. API Key Authentication](#)
- [9. S3 Upload Service](#)
- [10. Testing Strategy](#)
- [11. AWS Infrastructure](#)
 - [11.1 S3 \(raw CV storage\)](#)
 - [11.2 Secrets Manager](#)
 - [11.3 Elastic Beanstalk \(backend\)](#)
 - [11.4 Application Load Balancer](#)
 - [11.5 EC2](#)
 - [11.6 CodePipeline \(backend deploy\)](#)
- [12. GitHub Actions \(Dev / Prod\)](#)
- [13. Frontend — Simple UI](#)
- [14. Environment Variables](#)
- [15. Requirements](#)
- [16. Assessment Requirement Checklist](#)

1. Project Overview

Goal: A backend service that accepts a candidate’s CV (PDF/DOCX/image), extracts structured data using a multimodal LLM, suggests concrete improvements to the CV, recommends relevant jobs, and returns links to job boards / learning resources to close skill gaps.

Item	Detail
Language	Python 3.12
API Framework	FastAPI
Database	MongoDB (Atlas or self-hosted)
LLM Orchestration	LangChain, output validated with Pydantic schemas (Python’s equivalent of Zod)
LLM Providers	OpenAI (<code>gpt-4o</code>) and Google Gemini (<code>gemini-1.5-pro</code>) — multimodal, pluggable
Auth	API Key (header-based)
Storage	AWS S3 (raw CV files)
Tests	Pytest (unit + integration), coverage via <code>pytest-cov</code>
CI/CD	GitHub Actions — separate <code>dev</code> and <code>prod</code> pipelines
Backend Hosting	AWS Elastic Beanstalk (behind an Application Load Balancer)
Frontend Hosting	S3 + CloudFront, deployed via CodePipeline
Secrets	AWS Secrets Manager

2. Architecture



Secrets Manager stores: `OPENAI_API_KEY`, `GEMINI_API_KEY`, `MONGODB_URI`, `API_KEY_SALT`, `AWS_S3_BUCKET`, injected into Elastic Beanstalk environment at boot via `.ebextensions` + `boto3` fetch.

3. Folder Structure

```

cv-recommender-api/
├── app/
│   ├── main.py                # FastAPI app entrypoint
│   ├── config.py              # Settings (pydantic-settings), pulls from Secrets Manager
│   ├── deps.py                # Dependency-injected DB client, API key auth
│   ├── auth/
│   │   └── api_key.py         # API key verification middleware
│   ├── models/
│   │   ├── candidate.py       # Mongo document models (Pydantic + Motor/PyMongo)
│   │   ├── analysis.py
│   │   └── job.py
│   ├── schemas/
│   │   ├── cv_schema.py       # Pydantic schema the LLM must return (structured CV)
│   │   ├── improvement_schema.py # Pydantic schema for suggested edits
│   │   └── job_match_schema.py # Pydantic schema for job recommendations
│   ├── routers/
│   │   ├── cv.py              # /cv/upload, /cv/{id}
│   │   ├── analysis.py        # /cv/{id}/analysis
│   │   └── jobs.py            # /jobs/recommendations
│   ├── services/
│   │   ├── s3_service.py      # Upload/download to S3
│   │   ├── llm_service.py     # LangChain chains (extraction, improvement, matching)
│   │   ├── job_search_service.py # Builds job-board search links (LinkedIn, Indeed, etc.)
│   │   └── mongo_service.py   # DB access layer
│   └── utils/
│       └── file_parsing.py     # PDF/DOCX -> text/image conversion helpers
├── tests/
│   ├── unit/
│   │   ├── test_llm_service.py
│   │   ├── test_s3_service.py
│   │   └── test_schemas.py
│   └── integration/
│       ├── test_cv_upload_flow.py
│       └── test_jobs_endpoint.py
├── .ebextensions/
│   └── 01_environment.config    # EB env config, pulls secrets at deploy
├── .github/
│   └── workflows/
│       ├── dev.yml
│       └── prod.yml
├── frontend/                   # Simple React UI (see Section 9)
├── Dockerfile
├── requirements.txt
├── pytest.ini
└── README.md

```

4. Data Models (MongoDB)

```
# app/models/candidate.py
from pydantic import BaseModel, Field
from datetime import datetime
from typing import Optional
from bson import ObjectId

class Candidate(BaseModel):
    id: Optional[str] = Field(default=None, alias="_id")
    full_name: str
    email: Optional[str] = None
    cv_s3_key: str
    original_filename: str
    uploaded_at: datetime = Field(default_factory=datetime.utcnow)
    status: str = "processing" # processing | analyzed | failed

class Config:
    populate_by_name = True
    arbitrary_types_allowed = True
```

```
# app/models/analysis.py
from pydantic import BaseModel
from typing import List
from datetime import datetime

class Analysis(BaseModel):
    candidate_id: str
    extracted_profile: dict # raw structured CV (see CVSchema)
    improvements: List[dict] # see ImprovementSchema
    job_matches: List[dict] # see JobMatchSchema
    created_at: datetime
```

5. LLM Output Schemas (Pydantic — Python's Zod)

```
# app/schemas/cv_schema.py
from pydantic import BaseModel, Field
from typing import List, Optional

class Experience(BaseModel):
    company: str
    title: str
    start_date: Optional[str] = None
    end_date: Optional[str] = None
    summary: str
    achievements: List[str] = Field(default_factory=list)

class Education(BaseModel):
    institution: str
    degree: str
    field_of_study: Optional[str] = None
    graduation_year: Optional[str] = None

class ExtractedCV(BaseModel):
    full_name: str
    email: Optional[str] = None
    phone: Optional[str] = None
    headline: Optional[str] = Field(None, description="Current title / summary line")
    skills: List[str]
    years_of_experience: float
    experience: List[Experience]
    education: List[Education]
    certifications: List[str] = Field(default_factory=list)
```

```

# app/schemas/improvement_schema.py
from pydantic import BaseModel
from typing import List

class Improvement(BaseModel):
    section: str          # e.g. "Summary", "Experience - Company X"
    issue: str             # what's wrong
    suggestion: str        # concrete rewrite / fix
    priority: str           # "high" | "medium" | "low"

class ImprovementReport(BaseModel):
    overall_score: int     # 0-100
    strengths: List[str]
    improvements: List[Improvement]

```

```

# app/schemas/job_match_schema.py
from pydantic import BaseModel
from typing import List

class SkillGap(BaseModel):
    skill: str
    resource_name: str
    resource_url: str

class JobMatch(BaseModel):
    job_title: str
    match_score: int       # 0-100
    reasoning: str
    apply_links: List[str] # e.g. LinkedIn/Indeed search URLs
    skill_gaps: List[SkillGap] = []

class JobMatchReport(BaseModel):
    matches: List[JobMatch]

```

6. LangChain Service (Multimodal Extraction + Structured Output)

```

# app/services/llm_service.py
from langchain_openai import ChatOpenAI
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.messages import HumanMessage
from app.schemas.cv_schema import ExtractedCV
from app.schemas.improvement_schema import ImprovementReport
from app.schemas.job_match_schema import JobMatchReport
from app.config import settings
import base64

def get_llm(provider: str = "openai"):
    if provider == "gemini":
        return ChatGoogleGenerativeAI(model="gemini-1.5-pro", api_key=settings.GEMINI_API_KEY)
    return ChatOpenAI(model="gpt-4o", api_key=settings.OPENAI_API_KEY, temperature=0)

def extract_cv_from_file(file_bytes: bytes, mime_type: str, provider: str = "openai") -> ExtractedCV:
    """Multimodal call: sends the raw CV (PDF page images or DOCX-rendered images) to the LLM
    and forces structured output validated against the ExtractedCV Pydantic schema."""
    llm = get_llm(provider).with_structured_output(ExtractedCV)
    b64 = base64.b64encode(file_bytes).decode()

    message = HumanMessage(content=[
        {"type": "text", "text": "Extract all candidate information from this CV into the structured schema."},
        {"type": "image_url", "image_url": f"data:{mime_type};base64,{b64}"},
    ])
    return llm.invoke([message])

def generate_improvements(cv: ExtractedCV, provider: str = "openai") -> ImprovementReport:
    llm = get_llm(provider).with_structured_output(ImprovementReport)
    prompt = ChatPromptTemplate.from_messages([
        ("system", "You are a senior technical recruiter. Critique this CV honestly and "
        "give concrete, actionable rewrite suggestions per section."),
        ("human", "{cv_json}"),
    ])
    chain = prompt | llm
    return chain.invoke({"cv_json": cv.model_dump_json()})

def match_jobs(cv: ExtractedCV, provider: str = "openai") -> JobMatchReport:
    llm = get_llm(provider).with_structured_output(JobMatchReport)
    prompt = ChatPromptTemplate.from_messages([
        ("system", "Given this candidate profile, suggest the 5 best-fit job titles. "
        "For each, provide a match score, reasoning, 2-3 real job-board search "
        "links (LinkedIn Jobs, Indeed, We Work Remotely – build a search-query URL, "
        "do not invent a specific job posting), and skill gaps with a real learning "
        "resource link (Coursera, freeCodeCamp, official docs) for each gap."),
        ("human", "{cv_json}"),
    ])
    chain = prompt | llm
    return chain.invoke({"cv_json": cv.model_dump_json()})

```

Job-board link builder (deterministic, not LLM-generated, to guarantee valid URLs):

```
# app/services/job_search_service.py
from urllib.parse import quote_plus

def build_search_links(job_title: str, location: str = "") -> dict:
    q = quote_plus(job_title)
    loc = quote_plus(location)
    return {
        "linkedin": f"https://www.linkedin.com/jobs/search/?keywords={q}&location={loc}",
        "indeed": f"https://www.indeed.com/jobs?q={q}&l={loc}",
        "weworkremotely": f"https://weworkremotely.com/remote-jobs/search?term={q}",
    }

def build_learning_link(skill: str) -> dict:
    q = quote_plus(skill)
    return {
        "coursera": f"https://www.coursera.org/search?query={q}",
        "freecodecamp": f"https://www.freecodecamp.org/news/search/?query={q}",
    }
```

7. API Endpoints

Method	Path	Description	Auth
POST	/cv/upload	Multipart upload of CV file → stores raw file in S3, creates Candidate doc, kicks off async analysis	API Key
GET	/cv/{candidate_id}	Get candidate metadata + status	API Key
GET	/cv/{candidate_id}/analysis	Get extracted profile + improvement report	API Key
POST	/jobs/recommendations	Body: {candidate_id} → returns JobMatchReport with real search links	API Key
GET	/health	Liveness/readiness probe for ALB	none

Sample Route

```

# app/routers/cv.py
from fastapi import APIRouter, UploadFile, File, Depends, BackgroundTasks
from app.deps import require_api_key, get_db
from app.services import s3_service, llm_service, mongo_service
from app.utils.file_parsing import to_image_bytes

router = APIRouter(prefix="/cv", tags=["cv"])

@router.post("/upload", dependencies=[Depends(require_api_key)])
async def upload_cv(background_tasks: BackgroundTasks, file: UploadFile = File(...), db=Depends(get_db)):
    raw = await file.read()
    s3_key = s3_service.upload_file(raw, file.filename)
    candidate = mongo_service.create_candidate(db, file.filename, s3_key)

    background_tasks.add_task(run_analysis_pipeline, candidate["_id"], raw, file.content_type, db)
    return {"candidate_id": str(candidate["_id"]), "status": "processing"}

def run_analysis_pipeline(candidate_id, raw_bytes, mime_type, db):
    image_bytes = to_image_bytes(raw_bytes, mime_type) # normalize PDF/DOCX -> page images
    cv = llm_service.extract_cv_from_file(image_bytes, "image/png")
    improvements = llm_service.generate_improvements(cv)
    jobs = llm_service.match_jobs(cv)
    mongo_service.save_analysis(db, candidate_id, cv, improvements, jobs)

```

8. API Key Authentication

```

# app/auth/api_key.py
from fastapi import Header, HTTPException, status
from app.config import settings
import hmac

async def require_api_key(x_api_key: str = Header(...)):
    if not hmac.compare_digest(x_api_key, settings.API_KEY):
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid API key")

```

Keys are issued per-client, hashed and stored in MongoDB (`api_clients` collection) for multi-tenant use, with the raw key compared using `hmac.compare_digest` to avoid timing attacks. For the assessment's scope a single shared key from Secrets Manager is sufficient; the `api_clients` collection is the natural extension point.

9. S3 Upload Service

```

# app/services/s3_service.py
import boto3, uuid
from app.config import settings

s3 = boto3.client("s3", region_name=settings.AWS_REGION)

def upload_file(file_bytes: bytes, original_filename: str) -> str:
    key = f"cv/{uuid.uuid4()}-{original_filename}"
    s3.put_object(Bucket=settings.S3_BUCKET, Key=key, Body=file_bytes)
    return key

def get_presigned_url(key: str, expires_in: int = 3600) -> str:
    return s3.generate_presigned_url(
        "get_object",
        Params={"Bucket": settings.S3_BUCKET, "Key": key},
        ExpiresIn=expires_in,
    )

```


10. Testing Strategy

- **Unit tests** mock `boto3`, `langchain` LLM clients, and MongoDB (via `mongomock` or `pytest-mongo`) to test business logic in isolation.
- **Integration tests** spin up a test MongoDB (via `mongomock` / `Testcontainers`) and use FastAPI's `TestClient`, hitting real routers end-to-end with LLM calls mocked.
- Coverage target: $\geq 80\%$, enforced in CI (`pytest --cov=app --cov-fail-under=80`).

```
# tests/unit/test_schemas.py
from app.schemas.cv_schema import ExtractedCV

def test_extracted_cv_requires_full_name():
    data = {
        "full_name": "Ama Owusu",
        "skills": ["Python", "FastAPI"],
        "years_of_experience": 3.5,
        "experience": [],
        "education": [],
    }
    cv = ExtractedCV(**data)
    assert cv.full_name == "Ama Owusu"
    assert cv.years_of_experience == 3.5
```

```
# tests/integration/test_cv_upload_flow.py
from fastapi.testclient import TestClient
from unittest.mock import patch
from app.main import app

client = TestClient(app)

@patch("app.services.s3_service.upload_file", return_value="cvs/fake-key.pdf")
def test_upload_cv_returns_processing_status(mock_upload):
    with open("tests/fixtures/sample_cv.pdf", "rb") as f:
        response = client.post(
            "/cv/upload",
            headers={"x-api-key": "test-key"},
            files={"file": ("sample_cv.pdf", f, "application/pdf")},
        )
    assert response.status_code == 200
    assert response.json()["status"] == "processing"
```

```
# pytest.ini
[pytest]
testpaths = tests
addopts = --cov=app --cov-report=term-missing --cov-fail-under=80
```

11. AWS Infrastructure

11.1 S3 (raw CV storage)

- Bucket: `cv-recommender-uploads-{env}` — versioning **on**, default encryption **SSE-S3**, block all public access.
- Lifecycle rule: transition to Glacier after 90 days (candidate documents are PII — set retention per compliance needs).

11.2 Secrets Manager

- Secret name: `cv-recommender/{env}/app-secrets` — JSON blob with `OPENAI_API_KEY`, `GEMINI_API_KEY`, `MONGODB_URI`, `API_KEY`.

- EC2 instance role for Elastic Beanstalk is granted `secretsmanager:GetSecretValue` scoped to this ARN only.

11.3 Elastic Beanstalk (backend)

- Platform: **Docker on Amazon Linux 2023**.
- Environment tier: Web server, load-balanced, min 2 / max 4 instances (Auto Scaling on CPU > 60%).
- `.ebextensions/01_environment.config` pulls secrets at instance boot:

```
# .ebextensions/01_environment.config
container_commands:
  01_fetch_secrets:
    command: "python3 scripts/load_secrets_to_env.py"
option_settings:
  aws:autoscaling:asg:
    MinSize: 2
    MaxSize: 4
  aws:elasticbeanstalk:environment:
    LoadBalancerType: application
  aws:elasticbeanstalk:healthreporting:system:
    SystemType: enhanced
```

11.4 Application Load Balancer

- Listener 443 (ACM cert) → target group on container port 8000.
- Health check path: `/health`.

11.5 EC2

- Instance type: `t3.small` (dev) / `t3.medium` (prod), inside a private subnet with NAT for outbound LLM API calls; ALB in public subnet.

11.6 CodePipeline (backend deploy)

1. **Source:** GitHub (via CodeStar connection), triggers on push to `main` (prod) / `develop` (dev).
2. **Build:** CodeBuild — `docker build`, run `pytest`, push image to ECR.
3. **Deploy:** Elastic Beanstalk deploy action, per-environment application versions.

12. GitHub Actions (Dev / Prod)

```
# .github/workflows/dev.yml
name: Deploy Dev
on:
  push:
    branches: [develop]
jobs:
  test-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install -r requirements.txt
      - run: pytest
      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${ secrets.AWS_ROLE_DEV }
          aws-region: eu-west-1
      - name: Deploy to Elastic Beanstalk (dev)
        uses: einaregilsson/beanstalk-deploy@v22
        with:
          aws_access_key: ${ secrets.AWS_ACCESS_KEY_ID }
          aws_secret_key: ${ secrets.AWS_SECRET_ACCESS_KEY }
          application_name: cv-recommender
          environment_name: cv-recommender-dev
          region: eu-west-1
          version_label: dev-${ github.sha }
          deployment_package: deploy.zip
```

```
# .github/workflows/prod.yml
name: Deploy Prod
on:
  push:
    branches: [main]
jobs:
  test-and-deploy:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install -r requirements.txt
      - run: pytest --cov=app --cov-fail-under=80
      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${ secrets.AWS_ROLE_PROD }
          aws-region: eu-west-1
      - name: Deploy to Elastic Beanstalk (prod)
        uses: einaregilsson/beanstalk-deploy@v22
        with:
          aws_access_key: ${ secrets.AWS_ACCESS_KEY_ID }
          aws_secret_key: ${ secrets.AWS_SECRET_ACCESS_KEY }
          application_name: cv-recommender
          environment_name: cv-recommender-prod
          region: eu-west-1
          version_label: prod-${ github.sha }
          deployment_package: deploy.zip
```

Both workflows gate the deploy step behind a **passing test suite** — a failing `pytest` run stops the pipeline before any AWS action executes.

13. Frontend — Simple UI

A minimal single-page React app (Vite + TypeScript is fine here even inside the Python folder's assessment, since the brief only mandates TS for the dietitian project's backend — the FE stack is shared tooling):

- **Upload screen:** drag-and-drop CV, calls `POST /cv/upload`, polls `GET /cv/{id}` until `status === "analyzed"`.
- **Results screen:** three tabs — *Extracted Profile*, *Improvement Report* (score + suggestions list), *Job Matches* (cards with match score, reasoning, and clickable apply/learning links).
- API key stored in a `.env` (`VITE_API_KEY`) and sent as `x-api-key` header via a small `axios` instance.

Deployment: 1. `npm run build` → static files in `dist/`. 2. **S3** bucket `cv-recommender-frontend-{env}` (static website hosting **disabled**; access only via CloudFront OAC). 3. **CloudFront** distribution in front of the bucket, custom domain via ACM + Route 53. 4. **CodePipeline:** Source (GitHub) → Build (CodeBuild: `npm ci && npm run build`) → Deploy (S3 sync + CloudFront invalidation action).

14. Environment Variables

Variable	Purpose
<code>MONGODB_URI</code>	Mongo connection string
<code>OPENAI_API_KEY</code>	OpenAI access
<code>GEMINI_API_KEY</code>	Gemini access
<code>API_KEY</code>	Shared API key for clients
<code>S3_BUCKET</code>	Raw CV storage bucket
<code>AWS_REGION</code>	AWS region

15. Requirements

```
fastapi==0.115.*
uvicorn[standard]==0.30.*
pydantic==2.*
pydantic-settings==2.*
motor==3.*           # async MongoDB driver
langchain==0.3.*
langchain-openai==0.2.*
langchain-google-genai==2.*
boto3==1.35.*
pypdf==5.*
pdf2image==1.17.*
python-multipart==0.0.*
pytest==8.*
pytest-cov==5.*
pytest-asyncio==0.24.*
mongomock==4.*
httpx==0.27.*
```

16. Assessment Requirement Checklist

Requirement	Where addressed
Python	Entire backend (Section 3-9)
Unit/Integration tests	Section 10
FastAPI	Section 3, 7

API Key Authentication	Section 8
MongoDB	Section 4, 9
GitHub Actions (Dev/Prod)	Section 12
Multimodal LLMs (Gemini/OpenAI)	Section 6
LangChain + structured schema (Pydantic)	Section 5, 6
AWS: CodePipeline, Elastic Beanstalk, S3, EC2, Load Balancing, Secrets Manager	Section 11
Frontend: CloudFront/S3, CodePipeline	Section 13
Simple UI	Section 13
CV improvement suggestions	Section 5 (<code>ImprovementReport</code>), Section 6
Job recommendations + apply/learning links	Section 5 (<code>JobMatchReport</code>), Section 6, Section 9 (link builder)